

Auth Module

Authentication, MFA, password management, and token refresh.

Base path: `/api/v1/auth`

1. Authentication Flow

NepalCRM uses a stateless JWT-based authentication system. Every user authenticates once and receives two tokens: an **access token** (short-lived) and a **refresh token** (long-lived).

Supported User Types

Type	Login Endpoint	Firm Code Required
Super Admin	POST <code>/api/v1/super-admin/login</code>	No
Internal Users (Employees)	POST <code>/api/v1/auth/login</code>	Yes
Clients	POST <code>/api/v1/auth/client/login</code>	Yes

2. Endpoints

POST `/api/v1/auth/login`

Internal User Login for employees and firm admins.

```
Request:
{
  "data": {
    "username": "admin@firm.com",
    "password": "Secure@123",
    "lawFirmCode": "CLAW"
  }
}

Response (no MFA):
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOi...",
    "refreshToken": "dGhpcyBpcyBh...",
    "tokenType": "Bearer",
    "expiresIn": 900,
```

```

    "mfaRequired": false,
    "userId": "uuid",
    "username": "admin@firm.com",
    "userType": "FIRM_USER",
    "roleCode": "FIRM_ADMIN",
    "firmCode": "CLAW",
    "firmName": "Apex Law Associates",
    "mfaEnabled": true
  }
}

Response (MFA required):
{
  "success": true,
  "data": {
    "mfaToken": "temp-token-for-mfa-step",
    "mfaRequired": true,
    "mfaSetupRequired": false,
    ...
  }
}

```

POST /api/v1/auth/client/login

Client Login for client portal access.

```

Request:
{
  "data": {
    "username": "client@email.com",
    "password": "Client@123",
    "lawFirmCode": "CLAW"
  }
}

Response: Same structure as internal login.

```

POST /api/v1/auth/refresh

All Users Get a new access token using a refresh token.

```

Request:
{
  "data": {
    "refreshToken": "dGhpcyBpcyBh..."
  }
}

Response: {
  "data": { "accessToken": "new-jwt", "refreshToken": "new-refresh", ... }
}

```

3. MFA (Multi-Factor Authentication)

MFA uses TOTP (Time-based One-Time Password) via authenticator apps like Google Authenticator or Authy.

Flow

1. User logs in → response includes `mfaRequired: true` and an `mfaToken`
2. If `mfaSetupRequired: true`, the response also includes a `mfaSecret` and `mfaQrUrl`
3. User scans QR code in authenticator app
4. User confirms setup via `POST /api/v1/auth/mfa/setup/confirm`
5. On subsequent logins, user enters TOTP code → `POST /api/v1/auth/mfa/validate`
6. On success, receives full access + refresh tokens

POST /api/v1/auth/mfa/setup/confirm

Request:

```
{
  "data": {
    "mfaToken": "temp-token-from-login",
    "totpCode": "123456"
  }
}
```

Response: Full JWT tokens (MFA is now enabled for this user)

POST /api/v1/auth/mfa/validate

Request:

```
{
  "data": {
    "mfaToken": "temp-token-from-login",
    "totpCode": "123456"
  }
}
```

Response: Full JWT tokens

Dev bypass: In development mode (`app.production=false`), TOTP code `123456` bypasses MFA validation. Never used in production.

POST /api/v1/auth/change-password

Required on first login (when `passwordChangeRequired: true` in login response).

```
Request:
{
  "data": {
    "mfaToken": "temp-token-from-login",
    "currentPassword": "Temp@123",
    "newPassword": "Secure@123",
    "confirmPassword": "Secure@123"
  }
}

Response: Full JWT tokens (password updated)
```

4. Security Tokens

Token Type	Purpose	Lifetime	Endpoints Allowed
Access Token (JWT)	Standard API auth	15 minutes	All authorized endpoints
Refresh Token	Get new access token	7 days	/auth/refresh only
MFA Token	MFA setup/validate	5 minutes	/mfa/setup/confirm , /mfa/validate , /change-password

5. Error Codes

Status	Message	Cause
401	Bad credentials	Invalid username or password
401	Account is deactivated	User has been deactivated by admin
403	Invalid or expired MFA token	MFA token expired or wrong TOTP code
429	Too many attempts	Rate limiting triggered (implementation-specific)